

ObjektinisProgramavimas-3

3.0

Sugeneruota Doxygen 1.17.0

1 Concept Index	1
1.1 Concepts	1
2 Klasės Indeksas	3
2.1 Klasės	3
3 Failo Indeksas	5
3.1 Failai	5
4 Concept Documentation	7
4.1 VectorElement Concept Reference	7
4.1.1 Concept definition	7
4.1.2 Smulkus aprašymas	7
5 Klasės Dokumentacija	9
5.1 Vector< T, Allocator > Klasė Šablonas	9
5.1.1 Smulkus aprašymas	13
5.1.2 Konstruktoriaus ir Destruktoriaus Dokumentacija	13
5.1.2.1 Vector() [1/9]	13
5.1.2.2 Vector() [2/9]	14
5.1.2.3 Vector() [3/9]	14
5.1.2.4 Vector() [4/9]	14
5.1.2.5 Vector() [5/9]	15
5.1.2.6 Vector() [6/9]	16
5.1.2.7 Vector() [7/9]	16
5.1.2.8 Vector() [8/9]	16
5.1.2.9 Vector() [9/9]	17
5.1.3 Metodų Dokumentacija	17
5.1.3.1 append_range()	17
5.1.3.2 assign() [1/3]	17
5.1.3.3 assign() [2/3]	19
5.1.3.4 assign() [3/3]	19
5.1.3.5 assign_range()	19
5.1.3.6 at() [1/2]	20
5.1.3.7 at() [2/2]	20
5.1.3.8 emplace()	20
5.1.3.9 emplace_back()	21
5.1.3.10 erase() [1/2]	21
5.1.3.11 erase() [2/2]	22
5.1.3.12 get_allocator()	22
5.1.3.13 insert() [1/5]	22
5.1.3.14 insert() [2/5]	23
5.1.3.15 insert() [3/5]	23
5.1.3.16 insert() [4/5]	24

5.1.3.17 insert() [5/5]	24
5.1.3.18 insert_range()	24
5.1.3.19 max_size()	25
5.1.3.20 operator=() [1/3]	25
5.1.3.21 operator=() [2/3]	25
5.1.3.22 operator=() [3/3]	26
5.1.3.23 push_back() [1/2]	26
5.1.3.24 push_back() [2/2]	26
5.1.3.25 reserve()	27
5.1.3.26 resize() [1/2]	27
5.1.3.27 resize() [2/2]	27
5.1.3.28 shrink_to_fit()	27
5.1.3.29 swap()	28
6 Failo Dokumentacija	29
6.1 Vector.h Failo Nuoroda	29
6.1.1 Smulkus aprašymas	30
6.1.2 Funkcijos Dokumentacija	30
6.1.2.1 erase()	30
6.1.2.2 erase_if()	31
6.1.2.3 swap()	31
6.2 Vector.h	32
Rodyklė	41

skyrius 1

Concept Index

1.1 Concepts

Here is a list of all documented concepts with brief descriptions:

VectorElement	
Sąvoka, reikalaujanti, kad tipas <code>T</code> būtų perkeliamas (<code>std::movable</code>)	7

skyrius 2

Klasės Indeksas

2.1 Klasės

Klasės, struktūros, sąjungos ir sąsajos su trumpais aprašymais:

Vector< T, Allocator >	
Dinaminio masyvo šabloninė klasė	9

skyrius 3

Failo Indeksas

3.1 Failai

Visų dokumentuotų failų sąrašas su trumpais aprašymais:

Vector.h	Dinaminio masyvo šabloninė klasė, analogiška <code>std::vector</code>	29
--------------------------	---	--------------------

skyrius 4

Concept Documentation

4.1 VectorElement Concept Reference

Sąvoka, reikalaujanti, kad tipas `T` būtų perkeliamas (`std::movable`).

```
#include <Vector.h>
```

4.1.1 Concept definition

```
template<typename T>  
concept VectorElement = std::movable<T>
```

4.1.2 Smulkus aprašymas

Sąvoka, reikalaujanti, kad tipas `T` būtų perkeliamas (`std::movable`).

Tik tokie tipai gali būti `Vector` elementais.

Template Parameters

<code>T</code>	Tikrinamas tipas.
----------------	-------------------

skyrius 5

Klasės Dokumentacija

5.1 Vector< T, Allocator > Klasė Šablonas

Dinaminio masyvo šabloninė klasė.

```
#include <Vector.h>
```

Vieši Tipai

- using **value_type** = T
Elementų tipas.
- using **allocator_type** = Allocator
Atminties valdytojo tipas.
- using **alloc_traits** = std::allocator_traits<Allocator>
allocator_traits sinoniminė deklaracija patogumui.
- using **size_type** = std::size_t
Dydžiui skirtas tipas be ženklų.
- using **difference_type** = std::ptrdiff_t
Atstumui tarp elementų skirtas tipas su ženklu.
- using **reference** = T&
Elemento nuorodos tipas.
- using **const_reference** = const T&
Konstantinės elemento nuorodos tipas.
- using **pointer** = typename alloc_traits::pointer
Rodyklės tipas (iš allocator_traits).
- using **const_pointer** = typename alloc_traits::const_pointer
Konstantinės rodyklės tipas (iš allocator_traits).
- using **iterator** = T*
Tiesioginio iteratoriaus tipas.
- using **const_iterator** = const T*
Konstantinio tiesioginio iteratoriaus tipas.
- using **reverse_iterator** = std::reverse_iterator<iterator>
Atvirkštinio iteratoriaus tipas.
- using **const_reverse_iterator** = std::reverse_iterator<const_iterator>
Konstantinio atvirkštinio iteratoriaus tipas.

Vieši Metodai

- **Vector** () noexcept(noexcept(Allocator()))
Numatytasis konstruktorius. Sukuria tuščią masyvą.
- **Vector** (const Allocator &a) noexcept
Konstruktorius su allocatoriumi. Sukuria tuščią masyvą.
- **Vector** (size_type n, const Allocator &a=Allocator())
Konstruktorius su pradiniu dydžiu (numatytoji inicializacija).
- **Vector** (size_type n, const T &val, const Allocator &a=Allocator())
Konstruktorius su pradiniu dydžiu ir reikšme.
- **Vector** (const Vector &other)
Kopijavimo konstruktorius.
- **Vector** (const Vector &other, const Allocator &a)
Kopijavimo konstruktorius su nurodytu allocatoriumi.
- **Vector** (Vector &&other) noexcept
Perkėlimo konstruktorius.
- **Vector** (Vector &&other, const Allocator &a)
Perkėlimo konstruktorius su nurodytu allocatoriumi.
- template<std::input_iterator InputIt>
Vector (InputIt first, InputIt last, const Allocator &a=Allocator())
Konstruktorius iš iteratorių diapazono.
- **Vector** (std::initializer_list< T > il, const Allocator &a=Allocator())
Konstruktorius iš inicializavimo sąrašo.
- ~**Vector** ()
Destruktorius. Sunaikina visus elementus ir atlaisvina atmintį.
- **Vector** & operator= (const Vector &other)
Kopijavimo priskyrimo operatorius.
- **Vector** & operator= (Vector &&other) noexcept
Perkėlimo priskyrimo operatorius.
- **Vector** & operator= (std::initializer_list< T > il)
Priskyrimo operatorius iš inicializavimo sąrašo.
- **operator std::vector< T > () const**
Tiesioginis konvertavimas į std::vector<T>. Sukuria kopiją visų elementų.
- **operator std::span< T > ()**
Tiesioginis konvertavimas į kintamąjį std::span<T>.
- **operator std::span< const T > () const**
Tiesioginis konvertavimas į konstantinį std::span<const T>.
- template<std::input_iterator InputIt>
void **assign** (InputIt first, InputIt last)
Pakeičia turinį elementais iš iteratorių diapazono.
- void **assign** (size_type n, const value_type &val)
Pakeičia turinį n kopijų reikšmės val.
- void **assign** (std::initializer_list< value_type > il)
Pakeičia turinį inicializavimo sąrašo elementais.
- template<std::ranges::input_range R>
void **assign_range** (R &&r)
Pakeičia turinį diapazono elementais (C++23).
- **allocator_type get_allocator** () const noexcept
Grąžina naudojamo atminties valdytojo kopiją.
- **reference operator[]** (size_type n)
Prieiga be ribų tikrinimo.

- **const_reference operator[]** (size_type n) const
Konstantinė prieiga be ribų tikrinimo.
- **reference at** (size_type n)
Prieiga su ribų tikrinimu.
- **const_reference at** (size_type n) const
Konstantinė prieiga su ribų tikrinimu.
- **reference front** ()
Nuoroda į pirmą elementą.
- **const_reference front** () const
Konstantinė nuoroda į pirmą elementą.
- **reference back** ()
Nuoroda į paskutinį elementą.
- **const_reference back** () const
Konstantinė nuoroda į paskutinį elementą.
- **value_type * data** () noexcept
Rodyklė į vidinę atmintį.
- **const value_type * data** () const noexcept
Konstantinė rodyklė į vidinę atmintį.
- **iterator begin** () noexcept
Iteratorius į pirmą elementą.
- **const_iterator begin** () const noexcept
Konstantinis iteratorius į pirmą elementą.
- **const_iterator cbegin** () const noexcept
Konstantinis iteratorius į pirmą elementą.
- **iterator end** () noexcept
Iteratorius po paskutinio elemento.
- **const_iterator end** () const noexcept
Konstantinis iteratorius po paskutinio elemento.
- **const_iterator cend** () const noexcept
Konstantinis iteratorius po paskutinio elemento.
- **reverse_iterator rbegin** () noexcept
Atvirkštinis iteratorius į paskutinį elementą.
- **const_reverse_iterator rbegin** () const noexcept
Konstantinis atvirkštinis iteratorius į paskutinį elementą.
- **const_reverse_iterator crbegin** () const noexcept
Konstantinis atvirkštinis iteratorius į paskutinį elementą.
- **reverse_iterator rend** () noexcept
Atvirkštinis iteratorius prieš pirmą elementą.
- **const_reverse_iterator rend** () const noexcept
Konstantinis atvirkštinis iteratorius prieš pirmą elementą.
- **const_reverse_iterator crend** () const noexcept
Konstantinis atvirkštinis iteratorius prieš pirmą elementą.
- **bool empty** () const noexcept
true jei masyvas tuščias.
- **size_type size** () const noexcept
Elementų skaičius.
- **size_type capacity** () const noexcept
Talpa be persikirstymo.
- **size_type max_size** () const noexcept
Maksimalus teoriškai galimas elementų skaičius.
- **void reserve** (size_type n)

- Rezervuoja atmintį bent n elementams. Jei talpa jau pakankama — nieko nevykdo.*
- void `shrink_to_fit` ()
Sumažina talpa iki `size()`.
- void `resize` (`size_type` sz)
Pakeičia dydį naudojant numatytąją inicializaciją.
- void `resize` (`size_type` sz, const `value_type` &val)
Pakeičia dydį pildant reikšme `val`.
- void `push_back` (const `value_type` &val)
Prideda kopiją masyvo gale.
- void `push_back` (`value_type` &&val)
Perkelia elementą į masyvo galą.
- template<typename... Args>
`reference` `emplace_back` (Args &&... args)
Sukuria elementą tiesiogiai masyvo gale.
- template<std::ranges::input_range R>
void `append_range` (R &&r)
Prideda diapazono elementus gale (C++23).
- void `pop_back` ()
Pašalina paskutinį elementą. Nieko nevykdo jei tuščias.
- void `clear` () noexcept
Sunaikina visus elementus. `capacity()` lieka nepakitusi.
- `iterator` `insert` (const `iterator` cpos, const `value_type` &val)
Įterpia vieno elemento kopiją.
- `iterator` `insert` (const `iterator` cpos, `value_type` &&val)
Įterpia perkeliama elementą.
- `iterator` `insert` (const `iterator` cpos, `size_type` n, const `value_type` &val)
Įterpia n kopijų reikšmės `val`.
- template<std::input_iterator InputIt>
`iterator` `insert` (const `iterator` cpos, InputIt first, InputIt last)
Įterpia elementus iš iteratorių diapazono.
- `iterator` `insert` (const `iterator` cpos, std::initializer_list< `value_type` > il)
Įterpia inicializavimo sąrašo elementus.
- template<std::ranges::input_range R>
`iterator` `insert_range` (const `iterator` cpos, R &&r)
Įterpia diapazono elementus (C++23).
- template<typename... Args>
`iterator` `emplace` (const `iterator` cpos, Args &&... args)
Sukuria elementą tiesiogiai nurodytoje pozicijoje.
- `iterator` `erase` (const `iterator` cpos)
Pašalina elementą pozicijoje `cpos`.
- `iterator` `erase` (const `iterator` cfirst, const `iterator` clast)
Pašalina elementus diapazone [`cfirst`, `clast`).
- void `swap` (`Vector` &other) noexcept
Keičia turinį su `other`. $O(1)$.
- bool `operator==` (const `Vector` &other) const
Lygybė — vienodi elementai ta pačia tvarka.
- auto `operator<=>` (const `Vector` &other) const
Trijų krypčių palyginimas (C++20). Leksikografinė tvarka.
- bool `operator!=` (const `Vector` &other) const
Nelygybė — `true` jei masyvai skiriasi.
- bool `operator<` (const `Vector` &other) const

Leksikografinė mažiau-už relacija.

- `bool operator<=` (const `Vector` &other) const

Leksikografinė mažiau-arba-lygu relacija.

- `bool operator>` (const `Vector` &other) const

Leksikografinė daugiau-už relacija.

- `bool operator>=` (const `Vector` &other) const

Leksikografinė daugiau-arba-lygu relacija.

5.1.1 Smulkus aprašymas

```
template<VectorElement T, typename Allocator = std::allocator<T>>
class Vector< T, Allocator >
```

Dinaminio masyvo šabloninė klasė.

Saugo elementus nuosekliai atminties bloke. Kai laisvos vietos nebelieka, masyvas automatiškai išplečiamas į naują bloką pagal augimo strategiją: `nauja talpa = max(3 * capacity(), 1)`, tačiau jei `needed > 3 * capacity()` — naudojamas tikslus `needed`, taip vengiant perteklinės atminties eikvojimo didesnių įterpimų metu.

5.1.1.0.1 Optimizacijos

- **Trivialių tipų optimizacijos:** `resize`, `reallocate`, `shift_right`, `erase` naudoja `memcpy`/`memmove` kai `T` yra trivialiai kopijuojamas.
- **Trivialiai sunaikinami tipai:** destruktorių ciklai praleidiami.
- **Šakų prognozavimas:** `[[likely]]`/`[[unlikely]]` karštuosiuose keliuose.
- **`shift_right`:** inicializuotoje srityje naudoja priskyrimo operatorių (ne konstruktorių), neinicializuotoje — `std::move_if_noexcept`.
- **`resize(n)`:** vieno argumento versija nereikalauja `T` kopijavimo.

Template Parameters

<i>T</i>	Elementų tipas; turi tenkinti <code>VectorElement</code> sąvoką.
<i>Allocator</i>	Atminties valdytojo tipas. Turi tenkinti C++ Allocator reikalavimus. Numatytasis — <code>std::allocator<T></code> . Leidžia naudoti pool ar arena allocatorius našumui gerinti.

5.1.2 Konstruktoriaus ir Destructoriaus Dokumentacija

5.1.2.1 Vector() [1/9]

```
template<VectorElement T, typename Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector (
    const Allocator & a) [inline], [explicit], [noexcept]
```

Konstruktorius su allocatoriumi. Sukuria tuščią masyvą.

Parametrai

<i>a</i>	Naudojamas atminties valdytojas.
----------	----------------------------------

5.1.2.2 Vector() [2/9]

```
template<VectorElement T, typename Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector (
    size_type n,
    const Allocator & a = Allocator()) [inline], [explicit]
```

Konstruktorius su pradiniu dydžiu (numatytoji inicializacija).

Nereikalauja T kopijavimo — naudoja tik numatytąjį konstruktorių. Tinkamas perkeliams tik tipo klasėms.

Parametrai

<i>n</i>	Pradinių elementų skaičius.
<i>a</i>	Naudojamas atminties valdytojas.

5.1.2.3 Vector() [3/9]

```
template<VectorElement T, typename Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector (
    size_type n,
    const T & val,
    const Allocator & a = Allocator()) [inline]
```

Konstruktorius su pradiniu dydžiu ir reikšme.

Parametrai

<i>n</i>	Pradinių elementų skaičius.
<i>val</i>	Pradinė visų elementų reikšmė.
<i>a</i>	Naudojamas atminties valdytojas.

5.1.2.4 Vector() [4/9]

```
template<VectorElement T, typename Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector (
    const Vector< T, Allocator > & other) [inline]
```

Kopijavimo konstruktorius.

Sukuria gilią kopiją kito `Vector` objekto. Allocatorius parenkamas per `allocator_traits::select_on_copy_construction`.

Parametrai

<i>other</i>	Kopijuojamas objektas.
--------------	------------------------

5.1.2.5 Vector() [5/9]

```
template<VectorElement T, typename Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector (
    const Vector< T, Allocator > & other,
    const Allocator & a) [inline]
```

Kopijavimo konstruktorius su nurodytu allocatoriumi.

Parametrai

<i>other</i>	Kopijuojamas objektas.
--------------	------------------------

<i>a</i>	Naudojamas atminties valdytojas.
----------	----------------------------------

5.1.2.6 Vector() [6/9]

```
template<VectorElement T, typename Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector (
    Vector< T, Allocator > && other) [inline], [noexcept]
```

Perkėlimo konstruktorius.

Perimamas *other* turinys ir allocatorius; po operacijos *other* tampa tuščias.

Parametrai

<i>other</i>	Perkeliamas objektas.
--------------	-----------------------

5.1.2.7 Vector() [7/9]

```
template<VectorElement T, typename Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector (
    Vector< T, Allocator > && other,
    const Allocator & a) [inline]
```

Perkėlimo konstruktorius su nurodytu allocatoriumi.

Jei allocatoriai visada lygūs (*is_always_equal*) arba lyg šiuo atveju — perima turinį be kopijų. Jei skiriasi — perkelia elementus po vieną naudojant *move* konstruktorius (ne kopiją).

Parametrai

<i>other</i>	Perkeliamas objektas.
<i>a</i>	Naudojamas atminties valdytojas.

5.1.2.8 Vector() [8/9]

```
template<VectorElement T, typename Allocator = std::allocator<T>>
template<std::input_iterator InputIt>
Vector< T, Allocator >::Vector (
    InputIt first,
    InputIt last,
    const Allocator & a = Allocator()) [inline]
```

Konstruktorius iš iteratorių diapazono.

Template Parameters

<i>InputIt</i>	Išvesties iteratoriaus tipas.
----------------	-------------------------------

Parametrai

<i>first</i>	Diapazono pradžia.
--------------	--------------------

<i>last</i>	Diapazono pabaiga (neįskaičiuojama).
<i>a</i>	Naudojamas atminties valdytojas.

5.1.2.9 Vector() [9/9]

```
template<VectorElement T, typename Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector (
    std::initializer_list< T > il,
    const Allocator & a = Allocator()) [inline]
```

Konstruktorius iš inicializavimo sąrašo.

Parametrai

<i>il</i>	Inicializavimo sąrašas, pvz., {1, 2, 3}.
<i>a</i>	Naudojamas atminties valdytojas.

5.1.3 Metodų Dokumentacija

5.1.3.1 append_range()

```
template<VectorElement T, typename Allocator = std::allocator<T>>
template<std::ranges::input_range R>
void Vector< T, Allocator >::append_range (
    R && r) [inline]
```

Prideda diapazono elementus gale (C++23).

Template Parameters

<i>R</i>	Įvesties diapazono tipas.
----------	---------------------------

Parametrai

<i>r</i>	Diapazonas.
----------	-------------

5.1.3.2 assign() [1/3]

```
template<VectorElement T, typename Allocator = std::allocator<T>>
template<std::input_iterator InputIt>
void Vector< T, Allocator >::assign (
    InputIt first,
    InputIt last) [inline]
```

Pakeičia turinį elementais iš iteratorių diapazono.

Template Parameters

<i>Input</i> ↔ <i>It</i>	Įvesties iteratoriaus tipas.
-----------------------------	------------------------------

Parametrai

<i>first</i>	Diapazono pradžia.
--------------	--------------------

<i>last</i>	Diapazono pabaiga (neįskaičiuojama).
-------------	--------------------------------------

5.1.3.3 assign() [2/3]

```
template<VectorElement T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::assign (
    size_type n,
    const value_type & val) [inline]
```

Pakeičia turinį *n* kopijų reikšmės *val*.

Parametrai

<i>n</i>	Elementų skaičius.
<i>val</i>	Reikšmė, kuria pildoma.

5.1.3.4 assign() [3/3]

```
template<VectorElement T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::assign (
    std::initializer_list< value_type > il) [inline]
```

Pakeičia turinį inicializavimo sąrašo elementais.

Parametrai

<i>il</i>	Inicializavimo sąrašas.
-----------	-------------------------

5.1.3.5 assign_range()

```
template<VectorElement T, typename Allocator = std::allocator<T>>
template<std::ranges::input_range R>
void Vector< T, Allocator >::assign_range (
    R && r) [inline]
```

Pakeičia turinį diapazono elementais (C++23).

Template Parameters

<i>R</i>	Įvesties diapazono tipas.
----------	---------------------------

Parametrai

<i>r</i>	Diapazonas, iš kurio paimami elementai.
----------	---

5.1.3.6 at() [1/2]

```
template<VectorElement T, typename Allocator = std::allocator<T>>
reference Vector< T, Allocator >::at (
    size_type n) [inline]
```

Prieiga su ribų tikrinimu.

Išimtys

<code>std::out_of_range</code>	Jei <code>n >= size()</code> .
--------------------------------	-----------------------------------

5.1.3.7 at() [2/2]

```
template<VectorElement T, typename Allocator = std::allocator<T>>
const_reference Vector< T, Allocator >::at (
    size_type n) const [inline]
```

Konstantinė prieiga su ribų tikrinimu.

Išimtys

<code>std::out_of_range</code>	Jei <code>n >= size()</code> .
--------------------------------	-----------------------------------

5.1.3.8 emplace()

```
template<VectorElement T, typename Allocator = std::allocator<T>>
template<typename... Args>
iterator Vector< T, Allocator >::emplace (
    const_iterator cpos,
    Args &&... args) [inline]
```

Sukuria elementą tiesiogiai nurodytoje pozicijoje.

Template Parameters

<code>Args</code>	Konstruktoriaus argumentų tipai.
-------------------	----------------------------------

Parametrai

<code>cpos</code>	Pozicija.
<code>args</code>	Argumentai <code>T</code> konstruktoriui.

Gražina

Iteratorius į sukurtą elementą.

5.1.3.9 `emplace_back()`

```
template<VectorElement T, typename Allocator = std::allocator<T>>
template<typename... Args>
reference Vector< T, Allocator >::emplace_back (
    Args &&... args) [inline]
```

Sukuria elementą tiesiogiai masyvo gale.

Template Parameters

<i>Args</i>	Konstruktoriaus argumentų tipai.
-------------	----------------------------------

Parametrai

<i>args</i>	Argumentai T konstruktoriui.
-------------	------------------------------

Gražina

Nuoroda į sukurtą elementą.

Pastaba

[[nodiscard]] pagal C++23 standartą.

5.1.3.10 `erase()` [1/2]

```
template<VectorElement T, typename Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::erase (
    const_iterator cfirst,
    const_iterator clast) [inline]
```

Pašalina elementus diapazone [cfirst, clast).

Trivialiai kopijuojamiems tipams naudoja memmove.

Parametrai

<i>cfirst</i>	Pirmas šalinamas.
<i>clast</i>	Po paskutinio šalinamo.

Gražina

Iteratorius į po diapazono buvusį elementą.

5.1.3.11 erase() [2/2]

```
template<VectorElement T, typename Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::erase (
    const_iterator cpos) [inline]
```

Pašalina elementą pozicijoje *cpos*.

Trivialiai kopijuojamiems tipams naudoja `memmove`.

Parametrai

<i>cpos</i>	Šalinamo elemento pozicija.
-------------	-----------------------------

Gražina

Iteratorius į po pašalintojo buvusį elementą.

5.1.3.12 get_allocator()

```
template<VectorElement T, typename Allocator = std::allocator<T>>
allocator_type Vector< T, Allocator >::get_allocator () const [inline], [noexcept]
```

Gražina naudojamo atminties valdytojo kopiją.

Gražina

Atminties valdytojas.

5.1.3.13 insert() [1/5]

```
template<VectorElement T, typename Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::insert (
    const_iterator cpos,
    const value_type & val) [inline]
```

Įterpia vieno elemento kopiją.

Parametrai

<i>cpos</i>	Pozicija prieš kurią įterpiama.
<i>val</i>	Įterpiama reikšmė.

Gražina

Iteratorius į įterptą elementą.

5.1.3.14 insert() [2/5]

```
template<VectorElement T, typename Allocator = std::allocator<T>>
template<std::input_iterator InputIt>
iterator Vector< T, Allocator >::insert (
    const_iterator cpos,
    InputIt first,
    InputIt last) [inline]
```

Įterpia elementus iš iteratorių diapazono.

Jei talpos pakanka — stumiama vietoje. Priešingu atveju — realokacija.

Template Parameters

<i>InputIt</i>	Iteratoriaus tipas.
----------------	---------------------

Parametrai

<i>cpos</i>	Pozicija.
<i>first</i>	Pradžia.
<i>last</i>	Pabaiga.

Gražina

Iteratorius į pirmą įterptą elementą.

5.1.3.15 insert() [3/5]

```
template<VectorElement T, typename Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::insert (
    const_iterator cpos,
    size_type n,
    const value_type & val) [inline]
```

Įterpia *n* kopijų reikšmės *val*.

Jei talpos pakanka — stumiama vietoje. Priešingu atveju — realokacija.

Parametrai

<i>cpos</i>	Pozicija prieš kurią įterpiama.
<i>n</i>	Kiekis.
<i>val</i>	Reikšmė.

Gražina

Iteratorius į pirmą įterptą elementą.

5.1.3.16 insert() [4/5]

```
template<VectorElement T, typename Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::insert (
    const_iterator cpos,
    std::initializer_list< value_type > il) [inline]
```

Įterpia inicializavimo sąrašo elementus.

Parametrai

<i>cpos</i>	Pozicija.
<i>il</i>	Inicializavimo sąrašas.

Gražina

Iteratorius į pirmą įterptą elementą.

5.1.3.17 insert() [5/5]

```
template<VectorElement T, typename Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::insert (
    const_iterator cpos,
    value_type && val) [inline]
```

Įterpia perkeliamą elementą.

Parametrai

<i>cpos</i>	Pozicija prieš kurią įterpiama.
<i>val</i>	Perkeliamas reikšmė.

Gražina

Iteratorius į įterptą elementą.

5.1.3.18 insert_range()

```
template<VectorElement T, typename Allocator = std::allocator<T>>
template<std::ranges::input_range R>
iterator Vector< T, Allocator >::insert_range (
    const_iterator cpos,
    R && r) [inline]
```

Įterpia diapazono elementus (C++23).

Jei talpos pakanka — stumiama vietoje. Priešingu atveju — realokacija.

Template Parameters

<i>R</i>	Diapazono tipas.
----------	------------------

Parametrai

<i>cpos</i>	Pozicija.
-------------	-----------

<i>r</i>	Diapazonas.
----------	-------------

Gražina

Iteratorius į pirmą įterptą elementą.

5.1.3.19 max_size()

```
template<VectorElement T, typename Allocator = std::allocator<T>>
size_type Vector< T, Allocator >::max_size () const [inline], [nodiscard], [noexcept]
```

Maksimalus teoriškai galimas elementų skaičius.

Apima tiek allocatoriaus ribą (`allocator_traits::max_size`), tiek `difference_type` maksimumą — taip užtikrinama, kad rodyklių aritmetika `end()` – `begin()` nepersidengtų.

Gražina

Maksimalus elementų skaičius.

5.1.3.20 operator=() [1/3]

```
template<VectorElement T, typename Allocator = std::allocator<T>>
Vector & Vector< T, Allocator >::operator= (
    const Vector< T, Allocator > & other) [inline]
```

Kopijavimo priskyrimo operatorius.

Atlieka gilią kopiją. Apsaugo nuo savęs priskyrimo. Jei `propagate_on_container_copy_assignment` — kopijuoja allocatorių.

Parametrai

<i>other</i>	Kopijuojamas objektas.
--------------	------------------------

Gražina

Nuoroda į šį objektą.

5.1.3.21 operator=() [2/3]

```
template<VectorElement T, typename Allocator = std::allocator<T>>
Vector & Vector< T, Allocator >::operator= (
    std::initializer_list< T > il) [inline]
```

Priskyrimo operatorius iš inicializavimo sąrašo.

Parametrai

<i>il</i>	Inicializavimo sąrašas.
-----------	-------------------------

Gražina

Nuoroda į šį objektą.

5.1.3.22 operator=() [3/3]

```
template<VectorElement T, typename Allocator = std::allocator<T>>
Vector & Vector< T, Allocator >::operator= (
    Vector< T, Allocator > && other) [inline], [noexcept]
```

Perkėlimo priskyrimo operatorius.

Perimamas `other` turinys; po operacijos `other` tampa tuščias. Jei `propagate_on_container_move_assignment` — perima allocatorių.

Parametrai

<i>other</i>	Perkeliamas objektas.
--------------	-----------------------

Gražina

Nuoroda į šį objektą.

5.1.3.23 push_back() [1/2]

```
template<VectorElement T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::push_back (
    const value_type & val) [inline]
```

Prideda kopiją masyvo gale.

Parametrai

<i>val</i>	Pridedama reikšmė.
------------	--------------------

5.1.3.24 push_back() [2/2]

```
template<VectorElement T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::push_back (
    value_type && val) [inline]
```

Perkelia elementą į masyvo galą.

Parametrai

<i>val</i>	Perkeliamas reikšmė.
------------	----------------------

5.1.3.25 reserve()

```
template<VectorElement T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::reserve (
    size_type n) [inline]
```

Rezervuoja atmintį bent *n* elementams. Jei talpa jau pakankama — nieko nevykdo.

Parametrai

<i>n</i>	Pageidaujama minimali talpa.
----------	------------------------------

5.1.3.26 resize() [1/2]

```
template<VectorElement T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::resize (
    size_type sz) [inline]
```

Pakeičia dydį naudojant numatytąją inicializaciją.

Nereikalauja *T* kopijavimo. Tinkamas *move-only* tipams.

Parametrai

<i>sz</i>	Naujas dydis.
-----------	---------------

5.1.3.27 resize() [2/2]

```
template<VectorElement T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::resize (
    size_type sz,
    const value_type & val) [inline]
```

Pakeičia dydį pildant reikšme *val*.

Parametrai

<i>sz</i>	Naujas dydis.
<i>val</i>	Reikšmė naujiems elementams.

5.1.3.28 shrink_to_fit()

```
template<VectorElement T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::shrink_to_fit () [inline]
```

Sumažina talpa iki *size()*.

Pastaba: pagal standartą ši operacija yra neprivaloma (hint), tačiau ši implementacija visada ją vykdo.

5.1.3.29 swap()

```
template<VectorElement T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::swap (
    Vector< T, Allocator > & other) [inline], [noexcept]
```

Keičia turinį su *other*. $O(1)$.

Jei `propagate_on_container_swap` — keičiasi ir allocatoriai.

Parametrai

<i>other</i>	Kitas masyvas.
--------------	----------------

Dokumentacija šiai klasei sugeneruota iš šio failo:

- [Vector.h](#)

skyrius 6

Failo Dokumentacija

6.1 Vector.h Failo Nuoroda

Dinaminio masyvo šabloninė klasė, analogiška `std::vector`.

```
#include <algorithm>
#include <compare>
#include <concepts>
#include <cstring>
#include <initializer_list>
#include <iterator>
#include <limits>
#include <memory>
#include <span>
#include <stdexcept>
#include <type_traits>
#include <vector>
```

Klasės

- class `Vector< T, Allocator >`
Dinaminio masyvo šabloninė klasė.

Concepts

- concept `VectorElement`
Sąvoka, reikalaujanti, kad tipas `T` būtų perkliamas (`std::movable`).

Funkcijos

- template<`VectorElement` T, typename Allocator>
void `swap` (`Vector`< T, Allocator > &a, `Vector`< T, Allocator > &b) noexcept
Keičia du `Vector` objektus. $O(1)$.
- template<`VectorElement` T, typename Allocator, typename U>
`Vector`< T, Allocator >::size_type `erase` (`Vector`< T, Allocator > &v, const U &value)
Pašalina visus elementus lygius `value` (C++20).
- template<`VectorElement` T, typename Allocator, typename Pred>
`Vector`< T, Allocator >::size_type `erase_if` (`Vector`< T, Allocator > &v, Pred pred)
Pašalina visus elementus tenkinančius predikato sąlygą (C++20).

6.1.1 Smulkus aprašymas

Dinaminio masyvo šabloninė klasė, analogiška `std::vector`.

Šiame faile apibrėžta `Vector<T, Allocator>` klasė — pilnavertis dinaminis masyvas, suderinamas su C++20/23 standartais. Palaikomi:

- pilnas iteratorių rinkinys (tiesioginiai ir atvirkštiniai),
- `emplace_back`, `insert`, `erase` su stipria išimčių garantija,
- C++20 trijų krypčių palyginimo operatorius (`<=>`),
- C++23 operacijos `append_range`, `assign_range`, `insert_range`,
- pasirinktiniai atminties valdytojai per `Allocator` šablono parametą,
- `[[nodiscard]]` ant `emplace_back` (C++23),
- trivialių tipų optimizacijos (`memcpy/memmove`) per `<type_traits>`,
- `shift_right` naudoja priskyrimo operatorių inicializuotoje srityje ir `std::move_if_noexcept` neinicializuotoje srityje,
- `grow` išsaugo tikslų dydį kai `min_capacity > 2 * capacity()`,
- `max_size()` apima ir `difference_type` ribą rodyklių aritmetikai,
- šakų prognozavimo užuominos (`[[likely]]`/`[[unlikely]]`) karštuosiuose keliuose (`push_back`, `emplace_back`).

Template Parameters

<i>T</i>	Elementų tipas; turi tenkinti <code>VectorElement</code> sąvoką.
<i>Allocator</i>	Atminties valdytojo tipas. Numatytasis — <code>std::allocator<T></code> .

6.1.2 Funkcijos Dokumentacija

6.1.2.1 erase()

```
template<VectorElement T, typename Allocator, typename U>
Vector< T, Allocator >::size_type erase (
    Vector< T, Allocator > & v,
    const U & value)
```

Pašalina visus elementus lygius `value` (C++20).

Template Parameters

<i>T</i>	Elementų tipas.
<i>Allocator</i>	Atminties valdytojo tipas.
<i>U</i>	Lyginamos reikšmės tipas.

Parametrai

<i>v</i>	Masyvas.
----------	----------

<i>value</i>	Šalinama reikšmė.
--------------	-------------------

Gražina

Pašalintų elementų skaičius.

6.1.2.2 erase_if()

```
template<VectorElement T, typename Allocator, typename Pred>
Vector< T, Allocator >::size_type erase_if (
    Vector< T, Allocator > & v,
    Pred pred)
```

Pašalina visus elementus tenkinančius predikato sąlygą (C++20).

Template Parameters

<i>T</i>	Elementų tipas.
<i>Allocator</i>	Atminties valdytojo tipas.
<i>Pred</i>	Predikato tipas.

Parametrai

<i>v</i>	Masyvas.
<i>pred</i>	Predikatas.

Gražina

Pašalintų elementų skaičius.

6.1.2.3 swap()

```
template<VectorElement T, typename Allocator>
void swap (
    Vector< T, Allocator > & a,
    Vector< T, Allocator > & b) [noexcept]
```

Keičia du `Vector` objektus. O(1).

Template Parameters

<i>T</i>	Elementų tipas.
<i>Allocator</i>	Atminties valdytojo tipas.

Parametrai

<i>a</i>	Pirmas masyvas.
----------	-----------------

<i>b</i>	Antras masyvas.
----------	-----------------

6.2 Vector.h

[Eiti į šio failo dokumentaciją.](#)

```

00001
00024
00025 #ifndef VECTOR_H
00026 #define VECTOR_H
00027
00028 #include <algorithm>
00029 #include <compare>
00030 #include <concepts>
00031 #include <cstring>
00032 #include <initializer_list>
00033 #include <iterator>
00034 #include <limits>
00035 #include <memory>
00036 #include <span>
00037 #include <stdexcept>
00038 #include <type_traits>
00039 #include <vector>
00040
00048 template <typename T>
00049 concept VectorElement = std::movable<T>;
00050
00074 template <VectorElement T, typename Allocator = std::allocator<T>
00075 class Vector {
00076 public:
00077     // Tipų sinonimai
00078
00080     using value_type           = T;
00082     using allocator_type       = Allocator;
00084     using alloc_traits         = std::allocator_traits<Allocator>;
00086     using size_type            = std::size_t;
00088     using difference_type      = std::ptrdiff_t;
00090     using reference            = T&;
00092     using const_reference      = const T&;
00094     using pointer              = typename alloc_traits::pointer;
00096     using const_pointer        = typename alloc_traits::const_pointer;
00098     using iterator             = T*;
00100     using const_iterator       = const T*;
00102     using reverse_iterator     = std::reverse_iterator<iterator>;
00104     using const_reverse_iterator = std::reverse_iterator<const_iterator>;
00105
00106     // Konstruktoriai
00107
00111     Vector() noexcept(noexcept(Allocator())) : alloc() { create(); }
00112
00118     explicit Vector(const Allocator& a) noexcept : alloc(a) { create(); }
00119
00129     explicit Vector(size_type n, const Allocator& a = Allocator())
00130         : alloc(a) { create_default(n); }
00131
00139     Vector(size_type n, const T& val, const Allocator& a = Allocator())
00140         : alloc(a) { create(n, val); }
00141
00150     Vector(const Vector& other)
00151         : alloc(alloc_traits::select_on_container_copy_construction(other.alloc))
00152     { create(other.begin(), other.end()); }
00153
00160     Vector(const Vector& other, const Allocator& a)
00161         : alloc(a) { create(other.begin(), other.end()); }
00162
00170     Vector(Vector&& other) noexcept
00171         : dat(other.dat), avail(other.avail), limit(other.limit),
00172         alloc(std::move(other.alloc))
00173     { other.dat = other.avail = other.limit = nullptr; }
00174
00185     Vector(Vector&& other, const Allocator& a) : alloc(a) {
00186         if constexpr (alloc_traits::is_always_equal::value) {
00187             steal(other);
00188         } else if (a == other.alloc) {
00189             steal(other);
00190         } else {
00191             // Skirtingi allocatoriai -- perkelti elementus (ne kopijuoti)
00192             create(std::make_move_iterator(other.begin()),
00193                   std::make_move_iterator(other.end()));
00194         }
00195     }
00196

```

```

00205     template <std::input_iterator InputIt>
00206     Vector(InputIt first, InputIt last, const Allocator& a = Allocator())
00207         : alloc(a) { create(first, last); }
00208
00215     Vector(std::initializer_list<T> il, const Allocator& a = Allocator())
00216         : alloc(a) { create(il.begin(), il.end()); }
00217
00221     ~Vector() { uncreate(); }
00222
00223     // Priskyrimo operatoriai
00224
00234     Vector& operator=(const Vector& other) {
00235         if (this != &other) {
00236             if constexpr (alloc_traits::propagate_on_container_copy_assignment::value) {
00237                 if (alloc != other.alloc) {
00238                     uncreate();
00239                     alloc = other.alloc;
00240                 }
00241             }
00242             uncreate();
00243             create(other.begin(), other.end());
00244         }
00245         return *this;
00246     }
00247
00257     Vector& operator=(Vector&& other) noexcept {
00258         if (this != &other) {
00259             uncreate();
00260             if constexpr (alloc_traits::propagate_on_container_move_assignment::value)
00261                 alloc = std::move(other.alloc);
00262             steal(other);
00263         }
00264         return *this;
00265     }
00266
00273     Vector& operator=(std::initializer_list<T> il) {
00274         uncreate();
00275         create(il.begin(), il.end());
00276         return *this;
00277     }
00278
00283     explicit operator std::vector<T>() const { return std::vector<T>(dat, avail); }
00284
00288     explicit operator std::span<T>() { return std::span<T>(dat, avail); }
00289
00293     explicit operator std::span<const T>() const { return std::span<const T>(dat, avail); }
00294
00295     // assign
00296
00304     template <std::input_iterator InputIt>
00305     void assign(InputIt first, InputIt last) {
00306         size_type n = static_cast<size_type>(std::distance(first, last));
00307         if (n <= capacity()) {
00308             size_type old_sz = size();
00309             if (n <= old_sz) {
00310                 std::copy(first, last, dat);
00311                 destroy_range(dat + n, avail);
00312             } else {
00313                 InputIt mid = std::next(first, static_cast<difference_type>(old_sz));
00314                 std::copy(first, mid, dat);
00315                 std::uninitialized_copy(mid, last, avail);
00316             }
00317             avail = dat + n;
00318         } else {
00319             uncreate();
00320             create(first, last);
00321         }
00322     }
00323
00330     void assign(size_type n, const value_type& val) {
00331         if (n <= capacity()) {
00332             // Reuse buffer: overwrite existing, construct/destroy the difference
00333             size_type old_sz = size();
00334             if (n <= old_sz) {
00335                 std::fill(dat, dat + n, val);
00336                 destroy_range(dat + n, avail);
00337             } else {
00338                 std::fill(dat, avail, val);
00339                 std::uninitialized_fill(avail, dat + n, val);
00340             }
00341             avail = dat + n;
00342         } else {
00343             uncreate();
00344             create(n, val);
00345         }
00346     }
00347

```

```

00353 void assign(std::initializer_list<value_type> il) {
00354     uncreate(); create(il.begin(), il.end());
00355 }
00356
00363 template <std::ranges::input_range R>
00364 void assign_range(R&& r) {
00365     uncreate();
00366     create(std::ranges::begin(r), std::ranges::end(r));
00367 }
00368
00369 // get_allocator
00370
00375 allocator_type get_allocator() const noexcept { return alloc; }
00376
00377 // Prieiga prie elementu
00378
00380 reference operator[](size_type n) { return dat[n]; }
00382 const_reference operator[](size_type n) const { return dat[n]; }
00383
00388 reference at(size_type n) {
00389     if (n >= size()) [[unlikely]]
00390         throw std::out_of_range("Vector::at: index out of range");
00391     return dat[n];
00392 }
00393
00398 const_reference at(size_type n) const {
00399     if (n >= size()) [[unlikely]]
00400         throw std::out_of_range("Vector::at: index out of range");
00401     return dat[n];
00402 }
00403
00405 reference front() { return dat[0]; }
00407 const_reference front() const { return dat[0]; }
00409 reference back() { return avail[-1]; }
00411 const_reference back() const { return avail[-1]; }
00413 value_type* data() noexcept { return dat; }
00415 const value_type* data() const noexcept { return dat; }
00416
00417 // Iteratoriai
00418
00420 iterator begin() noexcept { return dat; }
00422 const_iterator begin() const noexcept { return dat; }
00424 const_iterator cbegin() const noexcept { return dat; }
00426 iterator end() noexcept { return avail; }
00428 const_iterator end() const noexcept { return avail; }
00430 const_iterator cend() const noexcept { return avail; }
00431
00433 reverse_iterator rbegin() noexcept { return reverse_iterator(end()); }
00435 const_reverse_iterator rbegin() const noexcept { return const_reverse_iterator(end()); }
00437 const_reverse_iterator crbegin() const noexcept { return const_reverse_iterator(end()); }
00439 reverse_iterator rend() noexcept { return reverse_iterator(begin()); }
00441 const_reverse_iterator rend() const noexcept { return const_reverse_iterator(begin()); }
00443 const_reverse_iterator crend() const noexcept { return const_reverse_iterator(begin()); }
00444
00445 // Talpa
00446
00448 [[nodiscard]] bool empty() const noexcept { return dat == avail; }
00450 [[nodiscard]] size_type size() const noexcept { return static_cast<size_type>(avail - dat); }
00452 [[nodiscard]] size_type capacity() const noexcept { return static_cast<size_type>(limit - dat); }
00453
00463 [[nodiscard]] size_type max_size() const noexcept {
00464     return std::min(
00465         alloc_traits::max_size(alloc),
00466         static_cast<size_type>(std::numeric_limits<difference_type>::max())
00467     );
00468 }
00469
00476 void reserve(size_type n) {
00477     if (n > capacity()) [[unlikely]]
00478         reallocate(n);
00479 }
00480
00487 void shrink_to_fit() {
00488     if (limit != avail) {
00489         if (size() == 0) {
00490             raw_deallocate();
00491             dat = avail = limit = nullptr;
00492         } else {
00493             size_type sz = size();
00494             iterator new_dat = alloc_traits::allocate(alloc, sz);
00495             trivial_move_or_uninit_move(dat, avail, new_dat);
00496             destroy_range(dat, avail);
00497             raw_deallocate();
00498             dat = new_dat;
00499             avail = limit = dat + sz;
00500         }
00501     }

```

```

00502     }
00503
00511 void resize(size_type sz) {
00512     if (sz < size()) {
00513         destroy_range(dat + sz, avail);
00514         avail = dat + sz;
00515     } else if (sz > size()) {
00516         if (sz > capacity()) [[unlikely]] reallocate(sz);
00517         for (iterator it = avail; it != dat + sz; ++it)
00518             alloc_traits::construct(alloc, it);
00519         avail = dat + sz;
00520     }
00521 }
00522
00529 void resize(size_type sz, const value_type& val) {
00530     if (sz < size()) {
00531         destroy_range(dat + sz, avail);
00532         avail = dat + sz;
00533     } else if (sz > size()) {
00534         if (sz > capacity()) [[unlikely]] reallocate(sz);
00535         std::uninitialized_fill(avail, dat + sz, val);
00536         avail = dat + sz;
00537     }
00538 }
00539
00540 // Keitimo operacijos
00541
00546 void push_back(const value_type& val) {
00547     if (avail == limit) [[unlikely]] grow();
00548     alloc_traits::construct(alloc, avail++, val);
00549 }
00550
00555 void push_back(value_type&& val) {
00556     if (avail == limit) [[unlikely]] grow();
00557     alloc_traits::construct(alloc, avail++, std::move(val));
00558 }
00559
00568 template <typename... Args>
00569 reference emplace_back(Args&&... args) {
00570     if (avail == limit) [[unlikely]] grow();
00571     alloc_traits::construct(alloc, avail++, std::forward<Args>(args)...);
00572     return back();
00573 }
00574
00581 template <std::ranges::input_range R>
00582 void append_range(R&& r) {
00583     if constexpr (std::ranges::sized_range<R>)
00584         reserve(size() + static_cast<size_type>(std::ranges::distance(r)));
00585     for (auto&& elem : r)
00586         push_back(std::forward<decltype(elem)>(elem));
00587 }
00588
00592 void pop_back() {
00593     if (avail != dat) [[likely]]
00594         alloc_traits::destroy(alloc, --avail);
00595 }
00596
00600 void clear() noexcept {
00601     destroy_range(dat, avail);
00602     avail = dat;
00603 }
00604
00612 iterator insert(const_iterator cpos, const value_type& val) {
00613     return insert(cpos, size_type{1}, val);
00614 }
00615
00623 iterator insert(const_iterator cpos, value_type&& val) {
00624     size_type index = static_cast<size_type>(cpos - dat);
00625     if (avail == limit) [[unlikely]] grow(size() + 1);
00626     iterator pos = dat + index;
00627     shift_right(pos, 1);
00628     alloc_traits::construct(alloc, pos, std::move(val));
00629     ++avail;
00630     return pos;
00631 }
00632
00643 iterator insert(const_iterator cpos, size_type n, const value_type& val) {
00644     if (n == 0) return const_cast<iterator>(cpos);
00645     size_type index = static_cast<size_type>(cpos - dat);
00646
00647     if (size() + n <= capacity()) [[likely]] {
00648         iterator pos = dat + index;
00649         shift_right(pos, n);
00650         for (size_type i = 0; i < n; ++i)
00651             alloc_traits::construct(alloc, pos + i, val);
00652         avail += n;
00653         return pos;

```

```

00654     } else {
00655         Vector tmp(alloc);
00656         tmp.reallocate(grow_capacity(size() + n));
00657         std::uninitialized_move(dat, dat + index, tmp.dat);
00658         std::uninitialized_fill(tmp.dat + index, tmp.dat + index + n, val);
00659         std::uninitialized_move(dat + index, avail, tmp.dat + index + n);
00660         tmp.avail = tmp.dat + size() + n;
00661         swap(tmp);
00662         return dat + index;
00663     }
00664 }
00665
00677 template <std::input_iterator InputIt>
00678 iterator insert(const_iterator cpos, InputIt first, InputIt last) {
00679     size_type index = static_cast<size_type>(cpos - dat);
00680     size_type n = static_cast<size_type>(std::distance(first, last));
00681     if (n == 0) return dat + index;
00682
00683     if (size() + n <= capacity()) [[likely]] {
00684         iterator pos = dat + index;
00685         shift_right(pos, n);
00686         iterator out = pos;
00687         for (auto it = first; it != last; ++it)
00688             alloc_traits::construct(alloc, out++, *it);
00689         avail += n;
00690         return pos;
00691     } else {
00692         Vector tmp(alloc);
00693         tmp.reallocate(grow_capacity(size() + n));
00694         std::uninitialized_move(dat, dat + index, tmp.dat);
00695         std::uninitialized_move(first, last, tmp.dat + index);
00696         std::uninitialized_move(dat + index, avail, tmp.dat + index + n);
00697         tmp.avail = tmp.dat + size() + n;
00698         swap(tmp);
00699         return dat + index;
00700     }
00701 }
00702
00710 iterator insert(const_iterator cpos, std::initializer_list<value_type> il) {
00711     return insert(cpos, il.begin(), il.end());
00712 }
00713
00724 template <std::ranges::input_range R>
00725 iterator insert_range(const_iterator cpos, R&& r) {
00726     size_type index = static_cast<size_type>(cpos - dat);
00727
00728     if constexpr (std::ranges::sized_range<R>) {
00729         size_type n = static_cast<size_type>(std::ranges::distance(r));
00730         if (n == 0) return dat + index;
00731
00732         if (size() + n <= capacity()) {
00733             iterator pos = dat + index;
00734             shift_right(pos, n);
00735             iterator out = pos;
00736             for (auto&& elem : r)
00737                 alloc_traits::construct(alloc, out++,
00738                     std::forward<decltype(elem)>(elem));
00739             avail += n;
00740             return pos;
00741         } else {
00742             // reallocation path (same as before)
00743             Vector tmp(alloc);
00744             tmp.reallocate(grow_capacity(size() + n));
00745             std::uninitialized_move(dat, dat + index, tmp.dat);
00746             std::uninitialized_copy(std::ranges::begin(r),
00747                                     std::ranges::end(r),
00748                                     tmp.dat + index);
00749             std::uninitialized_move(dat + index, avail,
00750                                     tmp.dat + index + n);
00751             tmp.avail = tmp.dat + size() + n;
00752             swap(tmp);
00753             return dat + index;
00754         }
00755     } else {
00756         // Non-sized / single-pass: collect into a temporary Vector first,
00757         // then delegate to the sized overload
00758         Vector<T, Allocator> tmp_range(alloc);
00759         for (auto&& elem : r)
00760             tmp_range.push_back(std::forward<decltype(elem)>(elem));
00761         return insert_range(cpos, tmp_range);
00762     }
00763 }
00764
00773 template <typename... Args>
00774 iterator emplace(const_iterator cpos, Args&&... args) {
00775     size_type index = static_cast<size_type>(cpos - dat);
00776     if (avail == limit) [[unlikely]] grow(size() + 1);

```

```

00777         iterator pos = dat + index;
00778         shift_right(pos, 1);
00779         alloc_traits::construct(alloc, pos, std::forward<Args>(args)...);
00780         ++avail;
00781         return pos;
00782     }
00783
00792     iterator erase(const_iterator cpos) {
00793         iterator pos = const_cast<iterator>(cpos);
00794         if constexpr (std::is_trivially_copyable_v<T>) {
00795             std::memmove(pos, pos + 1,
00796                 static_cast<std::size_t>(avail - pos - 1) * sizeof(T));
00797         } else {
00798             std::move(pos + 1, avail, pos);
00799         }
00800         alloc_traits::destroy(alloc, --avail);
00801         return pos;
00802     }
00803
00813     iterator erase(const_iterator cfirst, const_iterator clast) {
00814         iterator first = const_cast<iterator>(cfirst);
00815         iterator last = const_cast<iterator>(clast);
00816         iterator new_avail;
00817         if constexpr (std::is_trivially_copyable_v<T>) {
00818             std::size_t tail = static_cast<std::size_t>(avail - last);
00819             std::memmove(first, last, tail * sizeof(T));
00820             new_avail = first + tail;
00821         } else {
00822             new_avail = std::move(last, avail, first);
00823         }
00824         destroy_range(new_avail, avail);
00825         avail = new_avail;
00826         return first;
00827     }
00828
00836     void swap(Vector& other) noexcept {
00837         std::swap(dat, other.dat);
00838         std::swap(avail, other.avail);
00839         std::swap(limit, other.limit);
00840         if constexpr (alloc_traits::propagate_on_container_swap::value)
00841             std::swap(alloc, other.alloc);
00842     }
00843
00844     // Palyginimo operatoriai
00845
00849     bool operator==(const Vector& other) const {
00850         return size() == other.size() &&
00851             std::equal(begin(), end(), other.begin());
00852     }
00853
00857     auto operator<=>(const Vector& other) const {
00858         return std::lexicographical_compare_three_way(
00859             begin(), end(), other.begin(), other.end());
00860     }
00861
00863     bool operator!=(const Vector& other) const { return !(*this == other); }
00865     bool operator< (const Vector& other) const { return (*this <=> other) < 0; }
00867     bool operator<= (const Vector& other) const { return (*this <=> other) <= 0; }
00869     bool operator> (const Vector& other) const { return (*this <=> other) > 0; }
00871     bool operator>= (const Vector& other) const { return (*this <=> other) >= 0; }
00872
00873 private:
00874     iterator dat = nullptr;
00875     iterator avail = nullptr;
00876     iterator limit = nullptr;
00877     Allocator alloc;
00878
00879     // Pagalbiniai metodai
00880
00882     void create() { dat = avail = limit = nullptr; }
00883
00887     void create(size_type n, const T& val) {
00888         if (n == 0) { dat = avail = limit = nullptr; return; }
00889         dat = alloc.allocate(n);
00890         limit = avail = dat + n;
00891         std::uninitialized_fill(dat, limit, val);
00892     }
00893
00899     void create_default(size_type n) {
00900         if (n == 0) { dat = avail = limit = nullptr; return; }
00901         dat = alloc.allocate(n);
00902         avail = limit = dat + n;
00903         for (iterator it = dat; it != avail; ++it) alloc_traits::construct(alloc, it);
00904     }
00905
00912     template <typename InputIt>
00913     void create(InputIt first, InputIt last) {

```

```

00914         size_type n = static_cast<size_type>(std::distance(first, last));
00915         if (n == 0) { dat = avail = limit = nullptr; return; }
00916         dat = alloc.allocate(n);
00917         if constexpr (std::is_trivially_copyable_v<T> && std::is_pointer_v<InputIt>) {
00918             std::memcpy(dat, first, n * sizeof(T));
00919             avail = limit = dat + n;
00920         } else {
00921             limit = avail = std::uninitialized_move(first, last, dat);
00922         }
00923     }
00924
00925 void uncreate() {
00926     if (dat) {
00927         destroy_range(dat, avail);
00928         raw_deallocate();
00929     }
00930     dat = avail = limit = nullptr;
00931 }
00932
00933 void raw_deallocate() {
00934     alloc.deallocate(dat, static_cast<size_type>(limit - dat));
00935 }
00936
00937 void destroy_range(iterator first, iterator last) noexcept {
00938     if constexpr (!std::is_trivially_destructible_v<T>) {
00939         for (iterator it = last; it != first; )
00940             alloc_traits::destroy(alloc, --it);
00941     }
00942 }
00943
00944 static void trivial_move_or_uninit_move(iterator src_first,
00945                                         iterator src_last,
00946                                         iterator dst) {
00947     if constexpr (std::is_trivially_copyable_v<T>) {
00948         std::memcpy(dst, src_first,
00949                     static_cast<std::size_t>(src_last - src_first) * sizeof(T));
00950     } else {
00951         std::uninitialized_move(src_first, src_last, dst);
00952     }
00953 }
00954
00955 void reallocate(size_type new_capacity) {
00956     iterator new_dat = alloc.allocate(new_capacity);
00957     size_type sz = size();
00958     trivial_move_or_uninit_move(dat, avail, new_dat);
00959     destroy_range(dat, avail);
00960     if (dat) raw_deallocate();
00961     dat = new_dat;
00962     avail = dat + sz;
00963     limit = dat + new_capacity;
00964 }
00965
00966 size_type grow_capacity(size_type needed) const noexcept {
00967     size_type cap = capacity();
00968     size_type tripled = (cap <= std::numeric_limits<size_type>::max() / 3)
00969         ? 3 * cap
00970         : std::numeric_limits<size_type>::max();
00971     tripled = std::max(tripled, size_type{1});
00972     return (needed <= tripled) ? tripled : needed;
00973 }
00974
00975 void grow(size_type min_capacity = 0) {
00976     reallocate(grow_capacity(std::max(min_capacity, size() + 1)));
00977 }
00978
00979 void steal(Vector& other) noexcept {
00980     dat = other.dat;
00981     avail = other.avail;
00982     limit = other.limit;
00983     other.dat = other.avail = other.limit = nullptr;
00984 }
00985
00986 void shift_right(iterator pos, size_type n) {
00987     if (pos == avail) return;
00988     if constexpr (std::is_trivially_copyable_v<T>) {
00989         std::memmove(pos + n, pos,
00990                     static_cast<std::size_t>(avail - pos) * sizeof(T));
00991         return;
00992     }
00993     size_type tail = static_cast<size_type>(avail - pos);
00994     if (n >= tail) {
00995         iterator src = pos, dst = pos + n;
00996         for (; src != avail; ++src, ++dst)
00997             alloc_traits::construct(alloc, dst, std::move_if_noexcept(*src));
00998         destroy_range(pos, avail);
00999     } else {

```

```

01058         iterator uninit_src = avail - n;
01059         iterator uninit_dst = avail;
01060         for (iterator it = uninit_src; it != avail; ++it, ++uninit_dst)
01061             alloc_traits::construct(alloc, uninit_dst,
01062                                     std::move_if_noexcept(*it));
01063         for (iterator it = uninit_src; it != pos; ) {
01064             --it;
01065             *(it + n) = std::move_if_noexcept(*it);
01066         }
01067     }
01068 }
01069 };
01070
01071 // Laisvosios funkcijos
01072
01081 template <VectorElement T, typename Allocator>
01082 void swap(Vector<T, Allocator>& a, Vector<T, Allocator>& b) noexcept { a.swap(b); }
01083
01094 template <VectorElement T, typename Allocator, typename U>
01095 typename Vector<T, Allocator>::size_type erase(Vector<T, Allocator>& v, const U& value) {
01096     auto it = std::remove(v.begin(), v.end(), value);
01097     auto n = static_cast<typename Vector<T, Allocator>::size_type>(v.end() - it);
01098     v.erase(it, v.end());
01099     return n;
01100 }
01101
01112 template <VectorElement T, typename Allocator, typename Pred>
01113 typename Vector<T, Allocator>::size_type erase_if(Vector<T, Allocator>& v, Pred pred) {
01114     auto it = std::remove_if(v.begin(), v.end(), pred);
01115     auto n = static_cast<typename Vector<T, Allocator>::size_type>(v.end() - it);
01116     v.erase(it, v.end());
01117     return n;
01118 }
01119
01120 #endif // VECTOR_H

```


Rodyklë

`append_range`
 Vector< T, Allocator >, [17](#)

`assign`
 Vector< T, Allocator >, [17](#), [19](#)

`assign_range`
 Vector< T, Allocator >, [19](#)

`at`
 Vector< T, Allocator >, [19](#), [20](#)

`emplace`
 Vector< T, Allocator >, [20](#)

`emplace_back`
 Vector< T, Allocator >, [20](#)

`erase`
 Vector< T, Allocator >, [21](#)
 Vector.h, [30](#)

`erase_if`
 Vector.h, [31](#)

`get_allocator`
 Vector< T, Allocator >, [22](#)

`insert`
 Vector< T, Allocator >, [22–24](#)

`insert_range`
 Vector< T, Allocator >, [24](#)

`max_size`
 Vector< T, Allocator >, [25](#)

`operator=`
 Vector< T, Allocator >, [25](#)

`push_back`
 Vector< T, Allocator >, [26](#)

`reserve`
 Vector< T, Allocator >, [26](#)

`resize`
 Vector< T, Allocator >, [27](#)

`shrink_to_fit`
 Vector< T, Allocator >, [27](#)

`swap`
 Vector< T, Allocator >, [27](#)
 Vector.h, [31](#)

`Vector`
 Vector< T, Allocator >, [13](#), [14](#), [16](#), [17](#)

`Vector< T, Allocator >`, [9](#)
 [append_range](#), [17](#)
 [assign](#), [17](#), [19](#)
 [assign_range](#), [19](#)
 [at](#), [19](#), [20](#)
 [emplace](#), [20](#)
 [emplace_back](#), [20](#)
 [erase](#), [21](#)
 [get_allocator](#), [22](#)
 [insert](#), [22–24](#)
 [insert_range](#), [24](#)
 [max_size](#), [25](#)
 [operator=](#), [25](#)
 [push_back](#), [26](#)
 [reserve](#), [26](#)
 [resize](#), [27](#)
 [shrink_to_fit](#), [27](#)
 [swap](#), [27](#)
 Vector, [13](#), [14](#), [16](#), [17](#)
 Vector.h, [29](#)
 [erase](#), [30](#)
 [erase_if](#), [31](#)
 [swap](#), [31](#)
 VectorElement, [7](#)